
Oliver Hauke

ist Mathematiker und IT-Referent im Kompetenzzentrum „Analyse und Auswertung“ des Statistischen Bundesamtes. Er verantwortet die Weiterentwicklung der technischen Infrastruktur des Forschungsdatenzentrums und berät das Netzwerk empirische Steuerforschung in der effizienten Nutzung der Systeme.

Annette Kristiansen

ist Ökonomin und Referentin im Referat „Unternehmenssteuern“ des Statistischen Bundesamtes. Sie beschäftigt sich mit der Zusammenführung der Unternehmenssteuerstatistiken und betreut das Netzwerk empirische Steuerforschung. Für ihre externe Promotion an der Universität Bremen untersucht sie die technologische Vielfalt multinationaler Unternehmen.

EFFIZIENTE ANALYSE GROSSER FORSCHUNGSDATEN MITHILFE VON R

Oliver Hauke, Annette Kristiansen

✦ **Schlüsselwörter:** Effiziente Analysen, R-Programmierung, Forschungsdaten, Unternehmenssteuerstatistik, Business-Tax-Panel, Netzwerk empirische Steuerforschung

ZUSAMMENFASSUNG

Das Forschungsdatenzentrum des Statistischen Bundesamtes baut seine technische Infrastruktur gezielt aus, um der Wissenschaft erweiterte Analysemöglichkeiten amtlicher Daten bereitzustellen. Seit November 2025 steht Forschenden exklusiver Zugang zu diesen erweiterten Systemressourcen in R und Stata zur Verfügung. R-basierte Tools – insbesondere Apache Arrow, Parquet und DuckDB – ermöglichen es, gängige Analysen großer Forschungsdatensätze in Echtzeit auszuführen. Am Beispiel des Business-Tax-Panel (2013 bis 2019) werden die erzielbaren Effizienzgewinne bei der Auswertung umfangreicher steuerstatistischer Daten im Rahmen des Netzwerks empirische Steuerforschung demonstriert.

✦ **Keywords:** *Efficient analysis – R programming – research data – corporate tax statistics – Business Tax Panel – empirical tax research network*

ABSTRACT

The Research Data Centre at the Federal Statistical Office is strategically expanding its technical infrastructure to provide researchers with enhanced analytical capabilities for official data. Since November 2025, researchers have had exclusive access to these extended system resources in R and Stata. R-based tools – particularly Apache Arrow, Parquet, and DuckDB – enable real-time analysis of large research datasets. Using the Business Tax Panel (2013–2019), this paper demonstrates the achievable efficiency gains in analysing large-scale tax statistics within the empirical tax research network.

Vergleich der Performanz

5.1 Benchmark-Setup

Um zwischen den in Abschnitt **?@sec-methoden** vorgestellten Datenverarbeitungsmethoden abzuwägen, haben wir deren Performanz in einem systematischen Benchmark-Experiment auf dem SAS-Server des Statistischen Bundesamtes und den neuen OnSite R- und Stata-Servern des FDZ gemessen. Weiterführende Benchmarks sowie der vollständig reproduzierbare Code sind auf [open-code.de](https://www.opencode.de) verfügbar¹.

Herangezogen wurden dafür simulierte Einzeldaten in verschiedenen Größen (0.1%/1%/10% von den insgesamt 80 Mio. Beobachtungen des BTP). Sie wurden aus öffentlich verfügbaren Informationen generiert, um die wesentlichen Strukturen des BTP für unsere Analyse künstlich nachzubilden. Abgrenzend zu den im FDZ bereitgestellten *Datenstrukturfiles* wurden die Testdaten nicht aus echten Mikrodaten abgeleitet. Der simulierte Datensatz ist ausschließlich für technische Testzwecke der nachfolgenden Anwendungsfälle bestimmt:

- 1. **Fallzahlen** (pro Statistik und Jahr)
- 2. **Regression** (Einflussfaktoren auf Verlustvorträge)

Um eine zuverlässige Performanz-Messung zu erhalten, wurde jede Datenverarbeitungsmethode dreifach angewandt. Der Vergleich erfolgt anhand der mittleren Messung der folgenden Zielgrößen:

- 1. **Arbeitsspeicher**-Beanspruchung der Auswertung,
- 2. **Laufzeit** der Auswertung (real verstrichene Zeit),
- 3. **Festplattenspeicher** der verwendeten Daten.

Die Zielgrößen sind in dieser Reihenfolge zu priorisieren, da ein übermäßiger Arbeitsspeicherverbrauch zu schwerwiegenden Abbrüchen führen kann, während längere Laufzeiten dennoch zu einem erfolgreichen Ergebnis führen können. Festplattenspeicher ist ausreichend vorhanden, aber unnötige Belegung sollte vermieden werden.

Für unsere Aufgaben muss nicht zwingend die

Abstufungen der Zielgrößen sind für unsere Aufgaben nicht gleichermaßen relevant.

Eine Methode, die die betrachteten Auswertungen in unter 10 Sekunden mit weniger als 4 GB Arbeitsspeicher durchführen kann, wird bereits als optimal bewertet.

¹www.opencode.de/

Tabelle 1

Versionen betrachteter R-Pakete

Paket	Version
data.table	1.17.8
tidyverse	tba
vroom	tba
arrow	20.0.0.2
duckdb	1.3.2
polars	1.0.1

Tabelle 2

Baseline anhand SAS und Stata (1% simuliertes BTP)

Software	Anwendung	Arbeitsspeicher	Laufzeit	Festplatte
----------	-----------	-----------------	----------	------------

Weitere Optimierungen sind in diesem Fall nicht erforderlich. Falls bereits die gewünschte Performanz erreicht wurde, rückt die Reduktion des Entwicklungsaufwand in den Vordergrund. Entsprechend sind Aspekte wie die einfache Anwendbarkeit und Nachvollziehbarkeit – selbst für unerfahrene Nutzende – ausschlaggebend.

Wir betrachten in R (Version 4.5.0) die folgenden R-Pakete gemeinsam mit den in Abschnitt **?@sec-methoden** beschriebenen Methoden (Variablenselektion, Datenaufteilung):

Wie in Abschnitt **?@sec-infrastruktur** beschrieben war es uns aus organisatorischen Gründen nicht möglich, die neueste Version aller Pakete zu verwenden. Die leichter nutzbaren Variationen {duckplyr} bzw. {tidytable} zu nutzen. Die Package wurden aber wenige Tage nach dem Experiment eingerichtet. Die Syntaxtranslation kann dabei die Performanz verschlechtern, sodass bei {duckdb} bzw. {data.table} zwischen Performanz und Code-Verständlichkeit abgewägt werden muss. {arrow} ist schon länger als {polars} im Statistischen Bundesamt verfügbar, sodass wir darin bereits mehr Erfahrungen einsetzen konnten.

5.2 Baseline

Klassisch werden die betrachteten Auswertungsaufgaben im Statischen Bundesamt hauptsächlich mit SAS und im FDZ mit Stata gelöst. Um die nachfolgenden Ergebnisse einzuordnen, haben wir die Performanzmessungen für 1% des simulierten BTP durchgeführt (800.000 Beobachtungen)in SAS und Stata durchgeführt:

Der Arbeitsspeicher des FDZ OnSite-Servers reicht nicht

aus, um das ganze BTP in Stata zu verarbeiten, sodass	50	1000	Fallzahl	{readr}	CSV	""
zwingend mit selektivem Einlesen gearbeitet werden muss.	51	1000	Fallzahl	{base}	CSV	""
# A tibble: 62 x 8	52	1000000	Regression	{arrow}	Parquet	"Date"
Beobachtungen	53	1000000	Regression	{arrow}	Parquet	"Date"
Anwendung	54	1000000	Regression	{dplyr}	CSV	"Vari"
Paket	55	100000	Regression	{fs::bytes}	Parquet	"Date"
Dateiformat	56	100000	Regression	{dplyr}	CSV	"Vari"
<dbl> <chr>	57	10000	Regression	{dplyr}	CSV	"Vari"
1 1000000 Fallzahl {arrow} Parquet	58	10000	Regression	{arrow}	Parquet	"Date"
2 1000000 Fallzahl {duckdb} Parquet	59	10000	Regression	{dplyr}	CSV	""
3 1000000 Fallzahl {arrow} Parquet	60	1000	Regression	{dplyr}	CSV	"Vari"
4 1000000 Fallzahl {arrow} Parquet	61	1000	Regression	{dplyr}	CSV	""
5 1000000 Fallzahl {arrow} Parquet	62	1000	Regression	{arrow}	Parquet	"Date"
6 1000000 Fallzahl {arrow} Parquet	# i 2	more variables: Arbeitsspeicher <fs::bytes>, Laufzeit				
7 1000000 Fallzahl {arrow} Parquet						
8 1000000 Fallzahl {data.table} CSV						
9 1000000 Fallzahl {polars} Parquet						
10 1000000 Fallzahl {data.table} CSV						
11 1000000 Fallzahl {data.table} CSV						
12 100000 Fallzahl {polars} Parquet						
13 100000 Fallzahl {duckdb} Parquet						
14 100000 Fallzahl {arrow} Parquet						
15 100000 Fallzahl {data.table} CSV						
16 100000 Fallzahl {arrow} Parquet						
17 100000 Fallzahl {arrow} Parquet						
18 100000 Fallzahl {arrow} Parquet						
19 100000 Fallzahl {arrow} Parquet						
20 100000 Fallzahl {data.table} CSV						
21 100000 Fallzahl {data.table} CSV						
22 10000 Fallzahl {polars} Parquet						
23 10000 Fallzahl {data.table} CSV						
24 10000 Fallzahl {arrow} Parquet						
25 10000 Fallzahl {readr} CSV						
26 10000 Fallzahl {arrow} Parquet						
27 10000 Fallzahl {duckdb} Parquet						
28 10000 Fallzahl {data.table} CSV						
29 10000 Fallzahl {arrow} Parquet						
30 10000 Fallzahl {arrow} Parquet						
31 10000 Fallzahl {data.table} CSV						
32 10000 Fallzahl {arrow} Parquet						
33 10000 Fallzahl {readr} CSV						
34 10000 Fallzahl {arrow} CSV						
35 10000 Fallzahl {data.table} CSV.GZ						
36 10000 Fallzahl {base} CSV						
37 1000 Fallzahl {data.table} CSV						
38 1000 Fallzahl {readr} CSV						
39 1000 Fallzahl {polars} Parquet						
40 1000 Fallzahl {data.table} CSV						
41 1000 Fallzahl {duckdb} Parquet						
42 1000 Fallzahl {arrow} Parquet						
43 1000 Fallzahl {arrow} Parquet						
44 1000 Fallzahl {arrow} CSV						
45 1000 Fallzahl {data.table} CSV.GZ						
46 1000 Fallzahl {data.table} CSV						
47 1000 Fallzahl {arrow} Parquet						
48 1000 Fallzahl {arrow} Parquet						
49 1000 Fallzahl {arrow} Parquet						

5.3 Hauptergebnisse für große Datenmengen

""	121.22G
""	4.59G
""	4.59G
Anhang 1: Variablenauswahl, {arrow} + {parquet}	4.59G
(bei BTP in duckdb) im Gesamtpaket die eindeutige	4.59G
beste Datenaufteilungsmethode für unsere Zwecke.	4.59G
""	4.59G
> Das geringe Arbeitsspeicherbedarfs für die weitere	4.59G
Optimierung und ist in unserer Auswertung nahe	4.59G
zu konstant abhängig von der Datenmenge, so	4.59G
dass die Methoden auf noch größere Datenmengen	4.59G
skalieren können. Ansonsten bieten {duckdb} und {polars}	4.59G
stabile Optimierung des Arbeitsspeichers.	4.59G
""	469.79M
> schnellste Laufzeit für große Datenmengen	469.79M
""	1.21G
> das Parquet Dateiformat reduziert den benötigten Fest-	469.79M
plattenspeicher massiv.	469.79M
""	469.79M
> beste Nutzererfahrung	1.21G
""	464.38M
> Leicht und verständlich - adaptiert nativ schöne	469.79M
{tidyverse} Syntax	469.79M
""	997.06M
> Flexibel: selektives Einlesen ist zwingend erforder-	461.09M
lich	461.09M
""	1.21G
> hohe Stabilität, fehlende Features können durch	1.21G
leicht in duckdb oder in den Arbeitsspeicher über-	402.31M
führt und dennoch ausgeführt werden	402.31M
""	1.21G
> kleiner Nachteil: Erfahrung notwendig, da erst Date-	123.83M
naufteilung/selektives Einlesen die maximale Perform-	123.83M
anz erreichen	45.87M
""	45.87M
Unerwarteterweise ist {polars} für das simulierte 10%	45.87M
BTP abgeschlagen (unter den schlechtesten Methoden)	45.87M
und für andere Datenmengen optimal schnell.	45.87M
""	123.83M
""	40.11M
""	40.11M
5.4 Erkenntnisse für kleine Datenmengen	51.03M
""	51.03M
""	45.87M
> Parquet ist die optimale Laufzeit	45.87M

- › 1e6: arrow > duckdb > DT+SEL > polars
- › 1e5: polars > duckdb > arrow > DT+SEL
- › 1e4: polars > data.table > arrow > readr
- › Dagegen ist das effizienteste betrachtete Datenformat über 1000 Beobachtungen eindeutig Parquet – sowohl in Speicherbedarf als auch in Laufzeit.
- › Datenaufteilung bietet nur high-performance Methoden ab einer gewissen Datenmenge Vorteile. Für kleine Mengen überwiegt der Overhead durch das Lesen mehrerer kleiner Dateien. Klassische Methoden werden durch eine Datenaufteilung nur negativ beeinflusst.
- › Die Ergänzung von selektivem Einlesen ist immer vorteilhaft. High-Performance Methoden können aber bei einer aufgeteilten Datenhaltung darauf verzichten.
- › Von den klassischen Methoden ist DT+SEL am effizientesten, aber für unseren größten Anwendungsfall. Ein geschickter DTler kann mit moderaten Datenmengen auch sehr effizient arbeiten. Anfängern ist eher {arrow} zu empfehlen.
 - › 5x langsamer als {arrow}
 - › SEL notwendig, da ohne 3x größer als die Eingangsdaten, dauert 2.2 min anstatt 23 Sec. -> schwierige Anwendung (Entscheidung relevanter Variablen muss vorab getroffen werden unter Beachtung der Arbeitsspeicher Restriktionen)
 - › SPLIT verschlechtert DT im Gegensatz zu arrow
 - › {data.table}+csv ist die beste "klassische" R-Methode und erreicht maximal 23.58s, in der größten Ordnung wie die nicht optimierte Arrow Performance. Das die beste gemessene Performance für CSV (aufgeteilte Parquet sind doppelt so schnell). D.h. ein {data.table} Profi, ist besser als ein schlechter Arrow Programmierer, der aber größeres Potenzial für Verbesserungen hat.
- › base+readr sind immer weit abgeschlagen

5.5 Ergebnisse

- › Für {data.table} ist die geschickte Programmierung noch viel wichtiger. Arrow wird höchstens langsam und beansprucht viel weniger RAM. Wohingegen data.table für den gesamten Datensatz (ohne Sel) das 600x an RAM belegt wie Arrow, damit viel schneller Abbricht und den schlechten Programmierer Abbrüche beschert.
 - › ohne Vorauswahl bricht DT ab, wenn das gesamte BTP verwendet werden soll. Arrow nur, wenn die Auswertung auch dazu führt,

dass wirklich alle Variablen verarbeitet werden.

- › mit DT lässt sich nicht so flexibel/schön arbeiten
- › Arrow verzeiht Fehler leichter als DT. Alleine durch die Datenaufteilung (vorgegeben durch die Datenbereitstellung, vorbereitet durch das FDZ), erreichen wir Count Ergebnisse in 11.73s, nur 5s über dem absoluten Optimum.
- › Wenn noch weniger RAM einsetzbar ist, benötigt Polars den wenigsten RAM, DuckDB nur geringfügig mehr

10 %: arrow + aufgeteilte parquet, duckdb, arrow variante
 1 %: polars + ein parquet, duckdb, arrow 0.1 %: polars + ein parquet, DT, arrow 0.01%: DT + csv, readr, polars

-> nicht eindeutig. Für kleinere Datenmengen mehr Auswahl und das vertraute auszuwählen. Für größere Datenmengen klare Empfehlung Arrow, w.g. einfacher Anwendung, Stabilität (ggü. Nutzerfehlern), Skalierbarkeit an die RAM-Grenze und darüber hinaus, höchste Performance für unsere Datenmengen+gute System Ressourcen.

-> für kleine Datenmengen macht chunking keinen Sinn.

Dateitypen:

Beide Anwendungen lassen sich in unter 20 Z. mit Arrow programmieren (verständlich lesbar) s. Opencode Repo.

RAM Wachstum Arrow, Polars, DuckDB, Data.table, readr, base mit und ohne sel.

512 im FDZ Obergrenze.

Gewinne ggü. Klassischen Methoden 7.X Min DT und sehr viel RAM im Default. Dennoch nicht komplett davon abraten.

5.6 Fazit (nach Hinten)

- › Empfehlung (insb. für unerfahrene Nutzende): Arrow, stabil, flexibel, einfach zu nutzen
- › Profi's in data.table, die sehr vertraut sind mit der Syntax, müssen nur in Randfällen wechseln. Falls die wirklich genutzten Variablenauswahl nicht durch den verfügbaren Arbeitsspeicher bedient werden können (dt brauchte ca. 3x RAM ggü. Festplatte).
- › neuste, beste, experimentell, bietet polars für moderate Datenmengen die effizienteste Variante
- › wer mit sql vertraut ist oder Features außerhalb des Scopes von Arrow braucht (auf großen Inputs), nutzt auch gerne duckdb

->

Tabelle 3

Benchmarkergebnisse für Berechnung der Fallzahl (10% simuliertes BTP)

Beobachtungen	Anwendung	Paket	Dateiformat	Optimierung	Festplatte	Arbeitsspeicher
1e+06	Fallzahl	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	45.46G	219.23M
1e+06	Fallzahl	{duckdb}	Parquet		45.99G	4.48M
1e+06	Fallzahl	{arrow}	Parquet	Variablenauswahl	45.99G	669.55M
1e+06	Fallzahl	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	44.84G	4.47M
1e+06	Fallzahl	{arrow}	Parquet		45.99G	219.7M
1e+06	Fallzahl	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	44.89G	4.48M
1e+06	Fallzahl	{arrow}	Parquet		45.99G	668.71M
1e+06	Fallzahl	{data.table}	CSV	Variablenauswahl	121.22G	407.68M
1e+06	Fallzahl	{polars}	Parquet		45.99G	3.13M
1e+06	Fallzahl	{data.table}	CSV		121.22G	312.92G
1e+06	Fallzahl	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	97.45G	471.61G
1e+05	Fallzahl	{polars}	Parquet		4.59G	3.17M
1e+05	Fallzahl	{duckdb}	Parquet		4.59G	4.56M
1e+05	Fallzahl	{arrow}	Parquet	Variablenauswahl	4.59G	261M
1e+05	Fallzahl	{data.table}	CSV	Variablenauswahl	12.11G	40.94M
1e+05	Fallzahl	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	4.54G	219.08M
1e+05	Fallzahl	{arrow}	Parquet		4.59G	260.2M
1e+05	Fallzahl	{arrow}	Parquet		4.59G	219.7M
1e+05	Fallzahl	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	4.48G	219.25M
1e+05	Fallzahl	{data.table}	CSV		12.11G	31.41G
1e+05	Fallzahl	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	9.74G	46.98G
1e+04	Fallzahl	{polars}	Parquet		469.79M	3.13M
1e+04	Fallzahl	{data.table}	CSV	Variablenauswahl	1.21G	4.2M
1e+04	Fallzahl	{arrow}	Parquet	Variablenauswahl	469.79M	220.03M
1e+04	Fallzahl	{readr}	CSV	Variablenauswahl	1.21G	8.85M
1e+04	Fallzahl	{arrow}	Parquet		469.79M	219.19M
1e+04	Fallzahl	{duckdb}	Parquet		469.79M	4.49M
1e+04	Fallzahl	{data.table}	CSV		1.21G	3.15G
1e+04	Fallzahl	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	464.38M	219.23M
1e+04	Fallzahl	{arrow}	Parquet		469.79M	219.7M
1e+04	Fallzahl	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	997.06M	4.71G
1e+04	Fallzahl	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	461.09M	219.23M
1e+04	Fallzahl	{readr}	CSV		1.21G	1.62G
1e+04	Fallzahl	{arrow}	CSV		1.21G	26.1M
1e+04	Fallzahl	{data.table}	CSV.GZ		402.31M	4.38G
1e+04	Fallzahl	{base}	CSV		1.21G	9.02G
1e+03	Fallzahl	{data.table}	CSV	Variablenauswahl	123.83M	539.57K
1e+03	Fallzahl	{readr}	CSV	Variablenauswahl	123.83M	2.54M
1e+03	Fallzahl	{polars}	Parquet		45.87M	3.13M
1e+03	Fallzahl	{data.table}	CSV		123.83M	325.99M
1e+03	Fallzahl	{duckdb}	Parquet		45.87M	4.49M
1e+03	Fallzahl	{arrow}	Parquet	Variablenauswahl	45.87M	215.77M
1e+03	Fallzahl	{arrow}	Parquet		45.87M	214.86M
1e+03	Fallzahl	{arrow}	CSV		123.83M	3.3M
1e+03	Fallzahl	{data.table}	CSV.GZ		40.11M	468.94M
1e+03	Fallzahl	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	99.63M	487.24M
1e+03	Fallzahl	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	51.03M	219.25M
1e+03	Fallzahl	{arrow}	Parquet		45.87M	219.7M
1e+03	Fallzahl	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	48.12M	219.1M
1e+03	Fallzahl	{readr}	CSV		123.83M	188.25M
1e+03	Fallzahl	{base}	CSV		123.83M	680.31M
1e+06	Regression	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	44.84G	745.92M